Universidad Autónoma de Baja California

Facultad de Ciencias Quimicas e Ingenieria

Ingeniería en Computación

Navarrete Pérez Andrés - 1284707

Proyecto de carrera

Taller 8



Propósito, alcance y línea base

El objetivo del Taller 8 es definir una arquitectura integral, coherente y verificable para continuar desarrollando ChiTito Smart Dispenser. El producto está dirigido a dos mascotas, Chiquín y Tito, y permite solicitar alimento en gramos desde una aplicación web adaptable a teléfono, vincular una porción a una mascota mediante RFID, supervisar niveles y condición del agua, gestionar recirculación e informar el estado del equipo.

Se conserva la alternativa técnica previamente seleccionada: ESP32 DevKit V1, comunicación Wi-Fi/MQTT con Mosquitto, backend Laravel, base de datos MySQL y frontend Vue 3 como aplicación web progresiva (PWA). La aplicación controla y consulta a través de Laravel; la lógica de movimiento y las protecciones inmediatas residen en el ESP32.

Funcionalidad incluida

- Seleccionar a Chiquín o Tito y solicitar una masa de alimento; pesar el plato mediante celda de carga de 5 kg y HX711; guardar gramos reales servidos y resultado.
- Tag RFID de la mascota registrada para identificar que mascota se alimenta.
- Estimar alimento restante con VL53L1X y agua disponible con A02YYUW; activar y detener una bomba de recirculación de 5 V con protección local por nivel.
- Observar TDS y turbidez como indicadores para avisos de mantenimiento, sin declarar potabilidad ni cantidad de agua consumida.
- Mostrar telemetría, estado de conexión, historial, alertas y controles autorizados desde Vue 3 PWA; preservar información y resultados en MySQL.

Límites y estado incremental

Quedan fuera del alcance congelado: cámara, reconocimiento visual o inteligencia artificial, aplicación móvil nativa independiente, análisis microbiológico, dosificación precisa de agua por mascota, cambio automático de agua, caudalímetro, sensor de presencia, finales de carrera, batería de respaldo y sensor DS18B20. El estado lógico de una salida no prueba movimiento físico del servo ni circulación de agua.

La primera etapa de implementación se enfoca en telemetría de peso bruto, distancia, porcentaje estimado de alimento y disponibilidad. RFID, compuertas, recirculación y monitoreo de condición del agua permanecen en la arquitectura objetivo para las etapas posteriores. La inclusión de un componente en el diseño no significa que ya esté instalado, calibrado o probado.

Decisiones que quedan congeladas

Aspecto	Decisión arquitectónica
Unidad embebida	Un ESP32 DevKit V1 ejecuta sensores, actuadores, control local y recuperación.
Comunicación	Wi-Fi 2.4 GHz; JSON sobre MQTT con Mosquitto y cifrado TLS.
Cadena de control	PWA → Laravel/HTTPS → trabajador MQTT → broker → ESP32.
Datos e interfaz	MySQL en red privada, consultado solamente por Laravel; Vue 3 PWA.
Seguridad operacional	Fallas de comunicación, sensor o reinicio conducen a salidas seguras; no se repiten órdenes de movimiento.

Diagrama general de arquitectura y despliegue

La arquitectura se divide en tres ámbitos. El navegador contiene la PWA; el VPS aloja Nginx, Laravel, el trabajador de mensajería, MySQL y Mosquitto; y la vivienda alberga el router y el dispositivo físico. El VPS puede ejecutar los servicios de aplicación y base de datos mediante Docker Compose, manteniendo los servicios internos aislados.

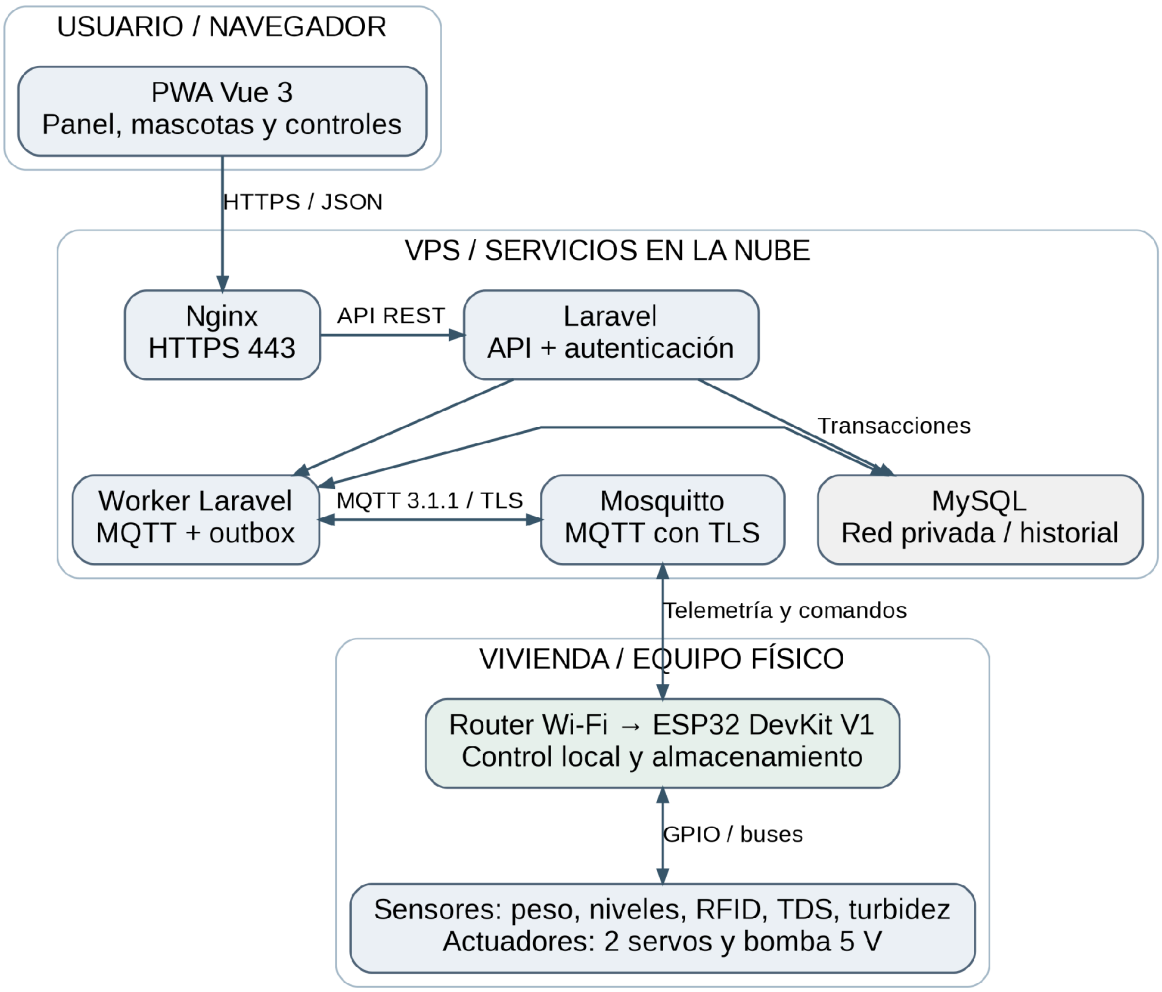


Figura 1. Arquitectura integral y distribución lógica de sus componentes.

Responsabilidades por nodo de despliegue

Nodo / ubicación	**Elementos**	**Acceso y responsabilidad**
Cliente (teléfono/PC)	Navegador + PWA Vue 3	Interfaz HTTPS, sesión y visualización; no posee credenciales MQTT ni conexión SQL.
VPS / entrada pública	Nginx + certificados TLS	Sirve la PWA y enruta /api a Laravel; entrada HTTPS 443.

Nodo / ubicación	Elementos	Acceso y responsabilidad
VPS / servicio privado	Laravel API + worker MQTT	Autenticación, reglas, outbox, publicación, consumo y ACK de eventos.
VPS / almacenamiento	MySQL + volúmenes persistentes	Escritura transaccional de datos, historial, comandos y eventos. Puerto 3306 no expuesto a Internet.
VPS / broker existente	Mosquitto	Transporta mensajes MQTT; listener cifrado para el ESP32 y ACL por identidad.
Vivienda	Router + ESP32 + periféricos	Adquisición, dosificación, RFID y protección de la bomba independientemente de la latencia remota.

Puertos y superficies previstos: 443/TCP para HTTPS; 8883/TCP como puerto MQTT sobre TLS si se utiliza el listener convencional; 3306/TCP únicamente en red privada de los servicios. La publicación del broker y su firewall deben validarse contra el despliegue real. El puerto 1883 sin TLS no se ofrece públicamente.

Diagrama de hardware y sistema eléctrico

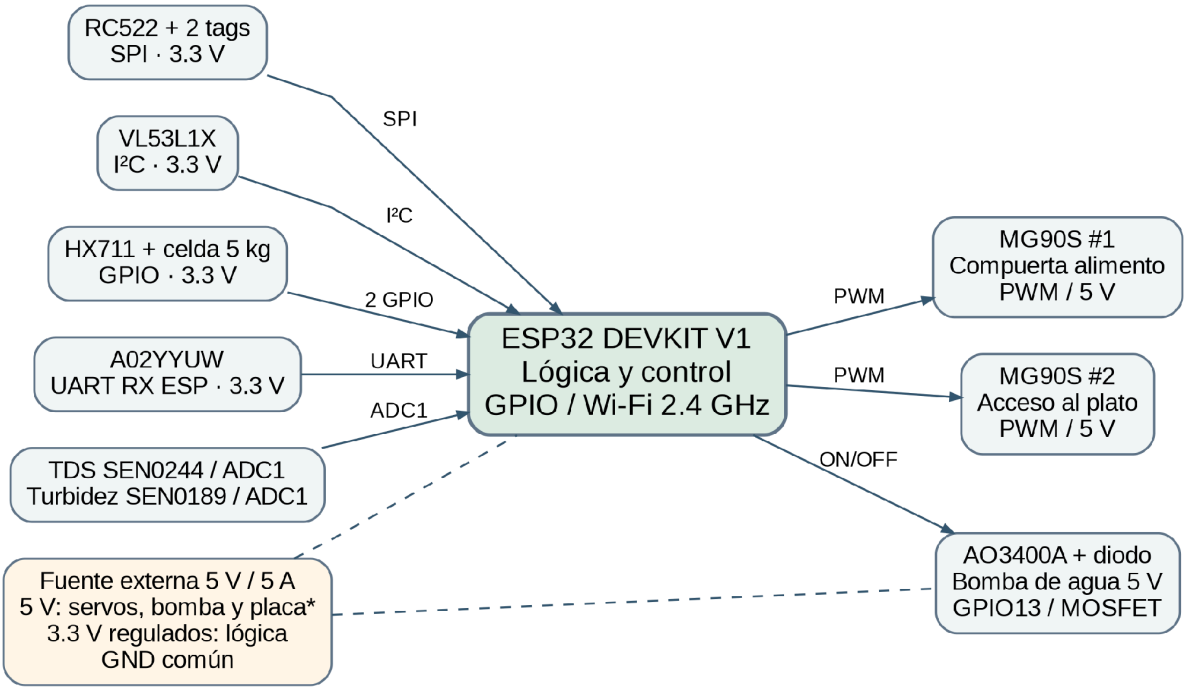


Figura 2. Arquitectura de hardware, buses de señal y alimentación. *La vía de 5 V hacia la placa depende del modelo exacto DevKit; no debe energizarse simultáneamente por USB y otra entrada sin verificar su circuito.

Componentes y función

Componente	Interfaz	Responsabilidad
ESP32 DevKit V1	Wi-Fi / GPIO	Control local, supervisión de fallas, comunicación y estado seguro.
HX711 + celda 5 kg	Digital 2 hilos	Peso estable del plato; diferencia entre peso inicial y final.
VL53L1X	I²C	Distancia para estimar porcentaje de alimento restante.

Componente	Interfaz	Responsabilidad
A02YYUW (SEN0311)	UART	Distancia para estimar nivel de agua; protección de bomba.
RC522 + 2 tags	SPI	Comparar UID de Chiquín o Tito con UID autorizado de la sesión.
2 × MG90S	PWM	Servo 1 dosifica alimento; servo 2 protege/accede al plato.
Bomba 5 V + AO3400A	Salida digital / potencia	Recirculación con encendido/parada y diodo de rueda libre.
SEN0244	ADC1	Estimación TDS; genera aviso de mantenimiento con calibración.
SEN0189	ADC1 + divisor	Señal analógica de turbidez; tendencia, no certificación de potabilidad.

Tabla de entradas y salidas (E/S)

Tipo	Origen o destino	Dato / operación
Entrada	Celda → HX711	Peso del plato en g, valor bruto y calidad de lectura.
Entrada	VL53L1X	Distancia en mm y estimación de nivel de alimento (%).
Entrada	A02YYUW	Distancia en mm y porcentaje de agua; lectura válida/reciente.

Tipo	Origen o destino	Dato / operación
Entrada	RC522	UID hexadecimal del tag leído; no acredita presencia física continua.
Entrada	SEN0244 / SEN0189	TDS estimado; lectura analógica de turbidez y calidad.
Entrada lógica	Mosquitto / MQTT	Comandos vigentes, configuración y confirmaciones de persistencia.
Salida física	MG90S #1	PWM: apertura/cierre de dosificador.
Salida física	MG90S #2	PWM: acceso al plato asociado a una sesión RFID.
Salida física	AO3400A + bomba	GPIO de encendido/apagado; apagado inmediato por seguridad.
Salida lógica	MQTT / backend	Telemetría, eventos, disponibilidad y resultados de comandos.

Asignación preliminar de pines y voltajes

El siguiente mapa integra las conexiones SPI, I²C y HX711 que ya se habían utilizado como referencia de programación y asigna provisionalmente los pines todavía libres. Es una propuesta, no un cableado definitivo: deberá verificarse la variante física de DevKit, el arranque y la alimentación de cada módulo antes del montaje.

Dispositivo / señal	GPIO ESP32	Alimentación / cuidado
RC522 SDA/SS (CS)	5	3.3 V; GPIO5 participa en strapping de arranque; asegurar CS no forzado en reset.
RC522 SCK / MISO / MOSI	18 / 19 / 23	SPI a 3.3 V. No aplicar lógica de 5 V.
RC522 RST	27	3.3 V; salida digital.
VL53L1X SDA / SCL	21 / 22	I²C con pull-ups a 3.3 V; alimentación del módulo según breakout.
HX711 DOUT / SCK	32 / 33	Módulo alimentado a 3.3 V para compatibilidad lógica.
A02YYUW TX → RX2	16	Sensor alimentado a 3.3 V, UART TTL; su pin RX selecciona modo según ficha.
MG90S dosificador / acceso	25 / 26	PWM a 3.3 V; potencia de servos en fuente externa 5 V.
AO3400A (gate)	13	GPIO a 3.3 V; resistencia de gate/pull-down; bomba externa a 5 V.
SEN0244 señal analógica	34 (ADC1)	Sensor a 3.3 V; salida típica 0–2.3 V.
SEN0189 señal acondicionada	35 (ADC1)	Módulo a 5 V; salida hasta 4.5 V: divisor a rango seguro ≤3.3 V.

Dispositivo / señal	GPIO ESP32	Alimentación / cuidado
UART0 de depuración	1 / 3	Reservados para programación y registro por USB; no asignados a sensores.

Precauciones eléctricas: los GPIO del ESP32 son de 3.3 V y no son entradas tolerantes a 5 V; GPIO34 y GPIO35 son solo entrada, por eso se destinan al ADC1; no utilizar ADC2 para estas lecturas mientras opera Wi-Fi. La fuente regulada de 5 V / 5 A alimenta las cargas, con tierras comunes, filtrado y desacoplo. El AO3400A trabaja como interruptor de lado bajo con diodo de protección para el motor, protección contra inversión de polaridad y polarización que mantenga la bomba apagada en arranque. La corriente real y el dimensionamiento del fusible deben medirse.

Alimentación del ESP32: seleccionar una única entrada permitida por su placa concreta (USB o pin 5 V, según serigrafía y circuito). Nunca alimentar directamente los servos o la bomba desde el pin 3V3 del ESP32. Los conectores mojados y las piezas móviles se aislarán físicamente de la electrónica.

Diagrama de módulos de software

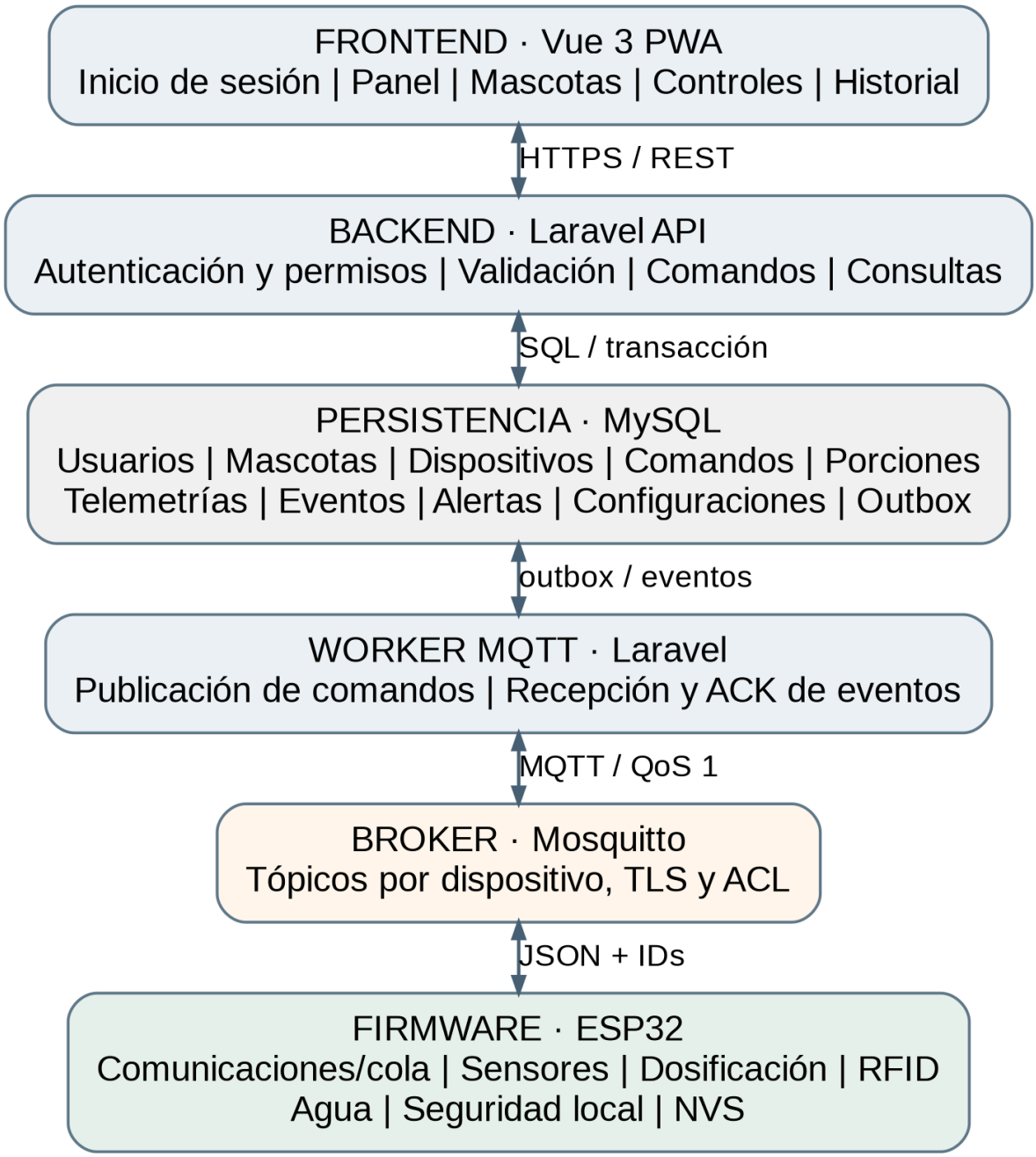


Figura 3. Módulos de frontend, backend, base de datos y firmware, separados por contratos de comunicación.

Firmware del ESP32

Módulo	Responsabilidad
FW-CORE: control y estados	Máquina concurrente de conectividad (C), alimento/acceso (A) y agua (W); prioridades y temporizadores sin bloqueo.
FW-SENSORES	Lecturas y calidad de HX711, VL53L1X, A02YYUW, SEN0244 y SEN0189; filtrado, compensaciones y calibración.
FW-ALIMENTO	Valida plato libre y peso estable, maneja MG90S #1, mide incremento real, cierra por umbral y registra sesión.
FW-RFID	Lee RC522, compara UID con la sesión, abre/cierra MG90S #2 con filtro de ausencia y evita accesos ajenos.
FW-AGUA	Calcula nivel, aplica histéresis, controla AO3400A y apaga la bomba por nivel, dato inválido o falla de enlace.
FW-MQTT	Wi-Fi, TLS, suscripción, publicación JSON, reconexión, command_id, event_id, lease y ACK.
FW-NVS	Guarda configuración y calibración, comandos recientes, sesión de porción, operación interrumpida y eventos no confirmados.

Backend Laravel y trabajador MQTT

Módulo	Responsabilidad
BE-AUTH / API	Inicio de sesión, autorización por usuario y dispositivo; validación de parámetros e idempotency_key.

Módulo	Responsabilidad
BE-DOMINIO	Reglas de mascotas, porciones, recirculación, restricciones de rango y estados de comandos.
BE-COMANDOS / OUTBOX	Transacción MySQL que crea comando y salida pendiente; publica después de COMMIT y recupera fallas.
BE-MQTT-WORKER	Consume telemetría/eventos, valida esquema y destino, deduplica event_id, persiste, emite ACK persisted.
BE-MONITOR	Supervisa heartbeat y backend lease, genera alertas y evita aceptar movimiento sin persistencia sana.

Modelo lógico de base de datos

MySQL conserva claves y relaciones entre usuario, dispositivo, mascota y ejecución. La estructura de dominio objetivo amplía las tablas técnicas del prototipo inicial; su creación por migraciones debe realizarse de manera incremental sin borrar historial existente.

Tabla	Contenido y regla principal
usuarios	Credenciales con hash; dueño autorizado de dispositivos.
mascotas	Chiquín y Tito; UID RFID activo único y relación con usuario.
dispositivos	device_id, propietario, última disponibilidad y configuración aplicada.
configuraciones	config_version, límites, tara, factores y umbrales versionados.
comandos	command_id UNIQUE, idempotency_key, acción, caducidad y estado.

Tabla	Contenido y regla principal
porciones	command_id UNIQUE, mascota, objetivo_g, servido_g real y resultado.
telemetrias	Peso, distancias, niveles, TDS/turbidez, calidad y fechas.
eventos	event_id UNIQUE, boot_id, seq, tipo, causa y referencia a comando.
alertas	Tipo, severidad, condición activa y atención por usuario.
salidas_pendientes	Outbox para publicar con seguridad después de la transacción.

Frontend Vue 3 PWA

La interfaz comprende inicio de sesión, panel principal con últimas lecturas y antigüedad, configuración de Chiquín/Tito y sus tags, solicitud de porciones, control de recirculación, historial y alertas, además de mantenimiento/configuración para el usuario autorizado. Un comando no se considera realizado por recibir HTTP 202; se debe consultar su estado hasta un resultado terminal persistido.

Flujo de datos y reglas de ejecución

Sensores y telemetría

Los módulos FW-SENSORES producen medidas con unidad, calidad, boot_id, seq y fecha o uptime. FW-MQTT publica en el tópico telemetry; el trabajador Laravel valida, almacena en MySQL y la API ofrece a la PWA las últimas muestras e historial. Si un sensor es inválido se transmite null y la causa, no el valor cero. Para la etapa de telemetría actual se conservan explícitamente los campos peso_raw, distancia_mm, alimento_pct y conexión.

Orden remota, ejecución local y confirmación

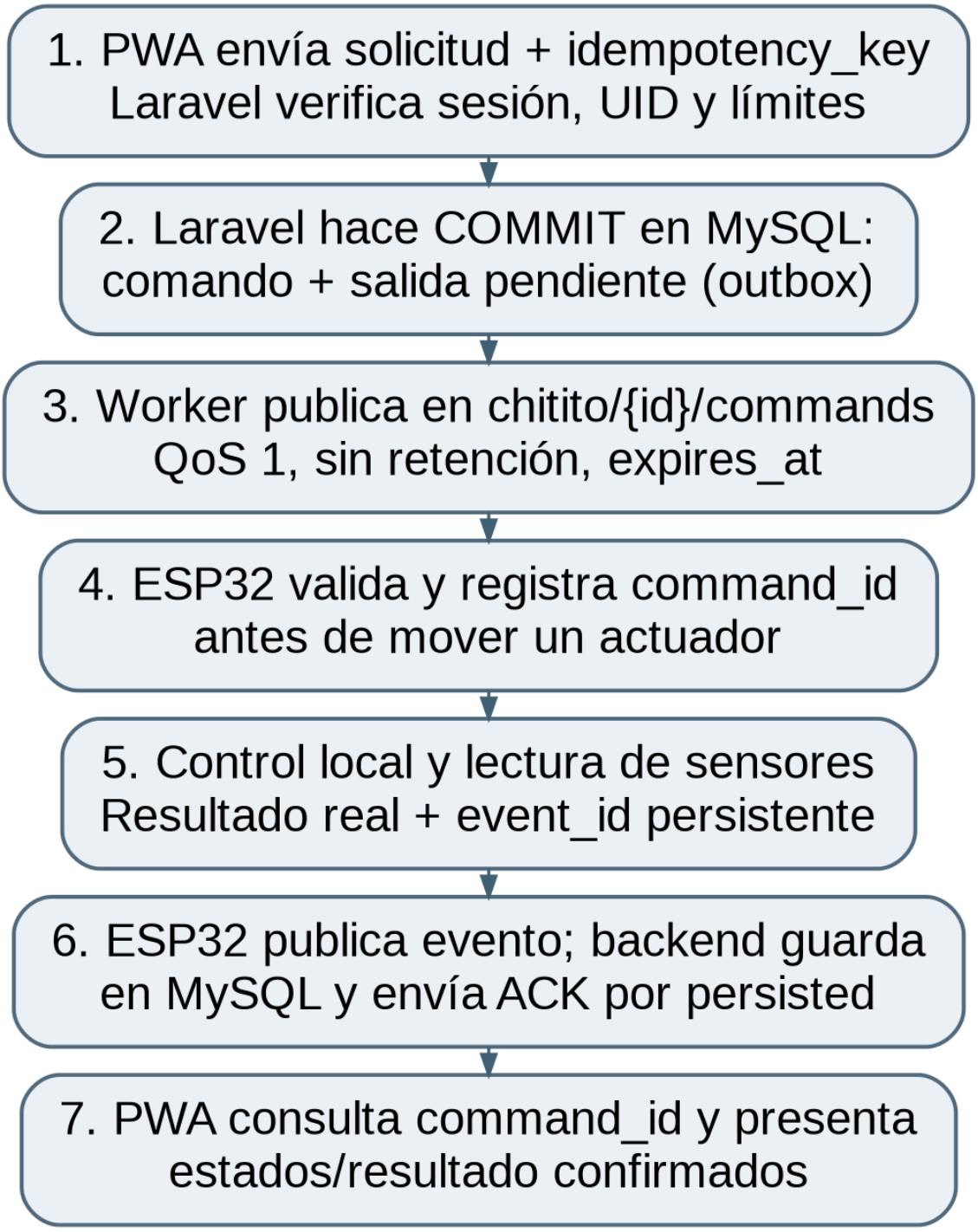


Figura 4. Flujo de comando y resultado. Un PUBACK MQTT no significa alimento servido ni evento persistido.

El control de alimento mide la diferencia entre el peso inicial y final estables del plato. La cantidad real servida nunca se sustituye por la solicitada ni se interpreta como consumo. Si hay una porción pendiente o restos, una nueva dosificación incompatible queda bloqueada; un error o una porción fuera de tolerancia no desencadena otra dosis automática. La tapa de acceso permanece cerrada mientras se prepara la porción.

La recirculación requiere una nueva orden de activación. El firmware usa nivel de agua reciente y válido, con histéresis (nivel_arranque_pct mayor que nivel_parada_pct). Si falta agua, falla el sensor o se pierde el enlace de control, la bomba se apaga sin esperar al servidor; recuperar agua o Internet no la enciende automáticamente.

Protocolos, tópicos y operaciones principales

Contratos y transportes

Enlace	Protocolo	Condición de uso
Navegador ↔ Nginx / Laravel	HTTPS, REST, JSON	TLS; sesión autenticada; PWA jamás accede a SQL ni al broker.
Laravel worker ↔ Mosquitto ↔ ESP32	MQTT 3.1.1 sobre TLS; QoS 1	Mensajes JSON; ACL por device_id; órdenes de movimiento sin retain.
ESP32 ↔ RC522	SPI, 3.3 V	SCK, MISO, MOSI, CS y reset; lectura de UID.
ESP32 ↔ VL53L1X	I²C, 3.3 V	Bus a través de GPIO21/22.
ESP32 ↔ A02YYUW	UART TTL, 3.3 V	Recepción de distancia en UART2 RX.
ESP32 ↔ HX711	Interfaz digital síncrona	DOUT y SCK; filtrado por firmware.
ESP32 ↔ sensores analógicos	ADC1, rango 0–3.3 V máximo	SEN0244 directo dentro de rango; SEN0189 con acondicionamiento.

Enlace	Protocolo	Condición de uso
ESP32 → servos / bomba	PWM / GPIO + etapa de potencia	Servos a 5 V; bomba con MOSFET, no conectada al GPIO.

Tópicos MQTT definidos

Tópico (id = device_id)	Sentido	Datos y política
chitito/{id}/telemetry	ESP32 → backend	Lecturas periódicas, calidad, salidas y estados.
chitito/{id}/events	ESP32 → backend	Resultados de comandos, porciones, accesos, fallas y alertas; event_id.
chitito/{id}/commands	Backend → ESP32	SERVIR, BOMBA_ON, BOMBA_OFF, PARAR, REVALIDAR. QoS 1; retain false.
chitito/{id}/config	Backend → ESP32	Límites y calibraciones versionados; nunca una orden de movimiento.
chitito/{id}/persisted	Backend → ESP32	ACK por event_id solo después de COMMIT MySQL.
chitito/{id}/availability	ESP32 → backend	Heartbeat, presencia y MQTT Last Will (disponible/no disponible).
chitito/{id}/backend_status	Backend → ESP32	Disponibilidad + lease; mensajes frescos, sin retención.

API REST preliminar

Los siguientes endpoints son el contrato previsto para completar el backend; no se afirma que todos estén desplegados. Se agrupan bajo /api/v1 para permitir evolución de formato sin modificar el flujo de seguridad.

Método / ruta	Operación
POST /api/v1/auth/login	Autenticar persona cuidadora y crear sesión.
POST /api/v1/auth/logout	Cerrar sesión actual.
GET /api/v1/devices	Listar únicamente dispositivos autorizados.
GET /api/v1/pets · POST /api/v1/pets	Consultar y registrar mascota con UID.
PATCH /api/v1/pets/{petId}	Modificar datos de mascota/UID con autorización.
GET /api/v1/devices/{id}/telemetry/latest	Consultar valores más recientes y su antigüedad.
GET /api/v1/devices/{id}/telemetry	Consultar serie temporal por fechas.
POST /api/v1/devices/{id}/commands	Solicitar SERVIR, BOMBA_ON, BOMBA_OFF, PARAR o REVALIDAR.
GET /api/v1/commands/{commandId}	Consultar estado y resultado del comando.
GET /api/v1/devices/{id}/portions	Historial de porciones asociadas a mascotas.
GET /api/v1/devices/{id}/alerts	Listado y estado de alertas.
PATCH /api/v1/alerts/{alertId}/ack	Marcar alerta atendida sin desactivar falla física.
GET /api/v1/devices/{id}/config	Consultar parámetros vigentes.
PUT /api/v1/devices/{id}/config	Actualizar versión de configuración y registrar auditoría.

Formato preliminar de mensajes JSON

Cada intercambio utiliza schema_version = 1, device_id estable y tamaños/campos validados. issued_at, expires_at y los sellos temporales válidos se expresan en UTC. La telemetría debe permitir measured_at = null cuando el reloj todavía no es confiable, preservando boot_id, seq y uptime_ms.

Telemetría: chitito/{id}/telemetry

Ejemplo ilustrativo de contrato; las cifras son datos ficticios para describir campos, no mediciones del prototipo:

Campo / estructura	Valor ilustrativo
schema_version / device_id	1 / "device01"
boot_id / seq / uptime_ms	"boot-A12" / 281 / 92040
measured_at / connectivity	null / "online"
readings.peso_raw / peso_plato_g	862443 / 35.8
readings.distancia_mm / alimento_pct	145 / 62.0
readings.agua_pct / tds_ppm	48.0 / null
readings.turbidez_adc / turbidez_v	1560 / 1.26
quality por sensor	"valida", "sin_dato" o "calibracion_pendiente"
state / outputs	C3-A0-W0 / {"bomba": false}

Comando: chitito/{id}/commands

Campo	Ejemplo / significado
schema_version	1
device_id	"device01"
command_id	"c5418f52-7d8e-4c0c-95ef-01d2e6c23301" (UUID)
action	"SERVIR" (alternativas: BOMBA_ON, BOMBA_OFF, PARAR, REVALIDAR)
issued_at / expires_at	UTC, emitidos por el backend; una orden caducada se rechaza.

Campo	Ejemplo / significado
config_version	Versión que el ESP32 debe tener aplicada.
params	{"pet_id": 1, "target_g": 60, "authorized_uid": "04A1B2C3"}

Respuesta HTTP a un nuevo comando: 202 Accepted con command_id y estado pendiente. La clave Idempotency-Key enviada por el frontend mantiene el mismo comando frente a reintentos de la misma acción.

Evento terminal y confirmación

Mensaje	Campos y política
events / evento de porción	event_id UUID, command_id, device_id, boot_id, seq, tipo="porcion_preparada", mascota_id, objetivo_g, servido_g medido o null, resultado, causa y fecha.
persisted / ACK backend	{"schema_version":1,"device_id":"device01","event_id":"...","stored_at":"...Z"}. Se publica únicamente después del COMMIT.
backend_status / lease	Sesión de backend, secuencia creciente, emitido/expira; no retenido. Mensajes atrasados no revalidan control.
availability	Online/Offline + marca temporal cuando existe; Last Will indica caída no anunciada de la sesión MQTT.

Estados de un comando: pendiente, aceptado, en_ejecucion, confirmado, rechazado, vencido, cancelado, error o sin_confirmacion. Un comando repetido se responde con su estado o resultado registrado, sin ejecutar de nuevo un servo. Los resultados de porción distinguen preparada, parcial, fuera_tolerancia, cancelada e interrumpida.

Autenticación, autorización y protección de secretos

Usuarios y dispositivos

Los usuarios ingresan mediante credenciales en la PWA y una sesión administrada por Laravel, preferentemente usando el mecanismo de cookies seguras, HttpOnly, SameSite y protección CSRF en el mismo origen HTTPS. Las contraseñas se almacenan con hash seguro, nunca reversibles. Cada petición comprueba que el usuario pertenece al dispositivo y puede operar con sus mascotas; las operaciones de mantenimiento pueden requerir una autorización adicional dentro de ese propietario.

El ESP32 no reutiliza la contraseña del usuario. Posee identidad MQTT propia (device_id más credenciales individuales), conexión TLS con verificación del servidor y ACL que solo autorizan sus tópicos: publica telemetry/events/availability y recibe commands/config/persisted/backend_status. El trabajador Laravel usa otra identidad con permisos delimitados. Se rechazan mensajes cuyo device_id o esquema no coincida con la identidad esperada.

Gestión de credenciales y variables

- Laravel y el trabajador obtendrán APP_KEY, DB_PASSWORD, MQTT_USERNAME/MQTT_PASSWORD, certificados y otros secretos de variables de entorno o secretos montados en el servidor; .env de producción no se incorpora al repositorio.
- El ESP32 almacenará parámetros de conectividad y credenciales provisionadas en memoria no volátil, sin imprimirlas en logs ni enviarlas como telemetría; el binario y el equipo físico deben tratarse como recursos sensibles.
- Se restringirá el acceso al VPS, a los volúmenes persistentes y a las copias de seguridad. TLS y ACL serán obligatorios para MQTT remoto; MySQL únicamente accesible por la red de aplicación.
- Se usarán archivos de ejemplo .env.example con variables sin valores secretos y un procedimiento de rotación/revocación de credenciales cuando se sustituya una placa o se pierda acceso.

Operación sin Internet, fallas y reinicio

Política segura y recuperación

Situación	Respuesta obligatoria	Después de recuperar
Pérdida Wi-Fi o MQTT	Cerrar dosificador, apagar bomba y pausar acceso; continuar lecturas y reconexión no bloqueante.	Reenviar eventos sin ACK; NO ejecutar comandos retenidos ni antiguos.
Mosquitto disponible; Laravel/MySQL no	El lease del backend vence y se bloquean nuevas dosis y el encendido.	Requiere consumidor y base de datos sanos para nuevo lease.
Reinicio/corte de energía	Al arrancar: bomba OFF, compuerta de alimento cerrada, acceso sin nueva apertura; leer diario local.	Si había movimiento, marcar interrumpida; revisar y revalidar explícitamente.
Nivel de agua bajo o inválido	Apagar AO3400A localmente, guardar causa y alerta.	El nivel recuperado no enciende la bomba: exige orden nueva.
HX711 inválido o peso sin progreso	Cerrar dosificador; servido_g es null si no es confiable; no hay repetición automática.	Revisar sensor, plato, calibración y residuos antes de otra porción.
PWA desconectada / sesión expirada	Mostrar última lectura con hora y bloquear controles; no encolar movimientos.	Consultar estado de comando en curso antes de un nuevo intento.

Persistencia e idempotencia

Antes de accionar la compuerta, el firmware registra command_id como recibido/en ejecución. Retiene configuración versionada, calibración, sesión RFID, registros recientes y una cola acotada de resultados/eventos con event_id; un evento termina su ciclo únicamente cuando el backend confirma persisted después del COMMIT. Las

notificaciones terminales tienen prioridad sobre telemetría repetitiva y, si no se puede registrar con seguridad una nueva dosis, dicha dosis se bloquea.

Se proponen como parámetros iniciales pendientes de validación: heartbeat cada 10 s, lease de control de 30 s, expiración de órdenes de movimiento a los 30 s y reintentos crecientes de reconexión hasta 30 s. Estos números no son mediciones ni garantías; forman parte de la configuración validable en el prototipo.

Tecnologías y versiones principales seleccionadas

Las versiones siguientes son líneas principales objetivo para congelar compatibilidad durante la integración. Su presencia exacta en el VPS y la placa se confirmará con inventario de despliegue; un cambio de versión mayor no se considerará silenciosamente equivalente.

Capa	Selección objetivo	Criterio
Microcontrolador	ESP32 DevKit V1 (ESP32-WROOM-32)	Soporta Wi-Fi, SPI, I²C, UART, ADC1 y PWM.
Firmware	Arduino IDE 2.x + Arduino-ESP32 Core 3.3.x (basado en ESP-IDF 5.5)	Conserva la línea de trabajo Arduino acordada; sin exigir versiones de bibliotecas secundarias.
Mensajería	MQTT 3.1.1 + Eclipse Mosquitto 2.x	QoS 1, TLS, ACL, Last Will y tópicos por equipo.
Backend	Laravel 12.x + PHP 8.3	API REST, autenticación, validación y worker; compatibilidad documentada.
Base de datos	MySQL 8.4 LTS	Persistencia transaccional y migraciones.
Interfaz	Vue 3.x + Vite 7.x; PWA responsive	Uso en navegador de móvil/PC sin aplicación nativa.

Capa	Selección objetivo	Criterio
Despliegue	Docker Compose 2.x + Nginx; Linux VPS	Servicios con red interna y TLS en ingreso; almacenamiento persistente.

Estas versiones son una referencia técnica y no requieren sustituir componentes ya funcionando antes de verificar su versión, compatibilidad y respaldo. El contrato arquitectónico es más estable que la versión menor de cada biblioteca.

Matriz requisito–componente responsable

Se reutilizan los identificadores RF01–RF18 de los modelos de comportamiento del proyecto. La siguiente matriz muestra el módulo principal y los participantes que deberán satisfacer cada requisito; sirve como guía de implementación y pruebas.

ID y requisito	Módulo principal	Colaboración / elemento
RF01 Autenticar usuario y permisos	BE-AUTH + FE-AUTH	MySQL usuarios; sesión
RF02 Registrar mascota y UID	BE-DOMINIO + FE-MASCOTAS	MySQL mascotas; FW-RFID
RF03 Elegir mascota y gramos	FE-ALIMENTO + BE-DOMINIO	Validación de rango y UID
RF04 Transmitir comandos remotos	BE-COMANDOS + BE-MQTT	Mosquitto; FW-MQTT
RF05 Dosificar alimento por peso	FW-ALIMENTO	HX711; MG90S #1
RF06 Registrar masa real y resultado	FW-ALIMENTO + BE-MQTT	MySQL porciones/eventos
RF07 Autorizar acceso por RFID	FW-RFID	RC522; MG90S #2
RF08 Estimar nivel de alimento	FW-SENSORES + FE-PANEL	VL53L1X; telemetrias

ID y requisito	Módulo principal	Colaboración / elemento
RF09 Proteger bomba por nivel de agua	FW-AGUA	A02YYUW; AO3400A
RF10 Controlar recirculación	FE-AGUA + BE-COMANDOS	FW-AGUA; bomba
RF11 Monitorear TDS/turbidez	FW-SENSORES + BE-MONITOR	SEN0244; SEN0189
RF12 Publicar y consultar telemetría	FW-MQTT + BE-MQTT	MySQL; FE-PANEL
RF13 Consultar historial por mascota	BE-API + FE-HISTORIAL	MySQL porciones/comandos
RF14 Emitir alertas y fallas	BE-MONITOR + FE-ALERTAS	FW-CORE; MySQL alertas
RF15 Estado seguro ante incidente	FW-CORE	FW-ALIMENTO; FW-AGUA
RF16 Reconexión y sincronización	FW-MQTT + FW-NVS	BE-MQTT; evento persisted
RF17 Idempotencia y caducidad	BE-COMANDOS + FW-MQTT	command_id; expires_at
RF18 Calibración y mantenimiento	FE-CONFIG + BE-DOMINIO	FW-SENSORES; FW-NVS

Verificación de arquitectura y control de cambios

Criterios de aceptación para la integración

- El panel recupera peso bruto, distancia, nivel calculado y última conexión a través de Laravel; la PWA no accede directamente a MySQL ni publica MQTT.
- Cada comando de dosificación genera un único command_id, tiene expiración y no puede ejecutar movimiento dos veces, incluso tras un reenvío o reinicio.
- El firmware cierra la compuerta por masa agregada realmente medida, produce resultado final estable y bloquea nuevas dosis si hay restos o una sesión incompatible.

- Solo el UID asociado a Chiquín o Tito para la porción preparada autoriza la apertura de acceso. La lectura de un tag distinto no abre.
- La bomba se apaga localmente por nivel bajo o inválido, pérdida de control y reinicio. El regreso del agua o de Internet no vuelve a encenderla.
- Un resultado en cola llega a MySQL una sola vez por event_id y recibe su ACK después del COMMIT; en la interfaz no se declara completado antes de ello.
- El prototipo conserva evidencia de calibraciones, alimentación y recuperación ante fallas; los resultados de pruebas no se inventan antes de ejecutarlas.

Línea base congelada y bitácora

Se establece la línea base A1 con fecha 20 de octubre de 2026. Quedan congelados: división PWA/Laravel/Mosquitto/ESP32/MySQL, interfaces de sensores y actuadores, modelos de porción y RFID, contrato de tópicos/IDs, reglas de seguridad, tablas de dominio previstas y política de operación sin enlace. Las posiciones de componentes, umbrales calibrados, rangos finales, valores exactos de tolerancia y pinout físico se consideran ajustes de ingeniería pendientes de validación, siempre que respeten los contratos congelados.

Campo de bitácora	Criterio para un cambio posterior
ID, fecha, autor	Registrar quién propone y cuándo se revisa.
Motivo / evidencia	Describir problema observado, limitación técnica o nueva necesidad.
Elementos afectados	Anotar RF, módulo, tabla, tópico, endpoint, pin o hardware.
Impacto y pruebas	Analizar compatibilidad, costo, seguridad, datos y pruebas de regresión.
Decisión	Aprobar o rechazar justificadamente; conservar versión A1 como referencia.

Conclusión

La arquitectura integral de ChiTito Smart Dispenser define una separación clara de responsabilidades: el ESP32 mide, dosifica y protege físicamente; Mosquitto transporta mensajes; Laravel autentica, verifica y coordina operaciones; MySQL conserva resultados reales; y Vue 3 permite supervisar y solicitar acciones desde cualquier navegador autorizado. Esta separación evita que una falla de Internet se convierta en una dosificación repetida o en una bomba funcionando sin nivel confiable.

El diseño mantiene la distinción esencial entre alimento solicitado, alimento realmente servido y alimento consumido, además de limitar los avisos de TDS y turbidez a mantenimiento. La línea base A1 permite continuar por etapas —partiendo de la telemetría ya definida— y evaluar cada componente mediante pruebas reales. Toda modificación posterior de estructura o contrato deberá justificarse y registrarse en la bitácora del proyecto.